

Customer Churn Prediction & Lifetime Value (LTV) System

Project Report

Prepared by: Kapish Gupta

Team Leader - Batch 14

Internship Project | Zaalima Development

Project Focus: An end-to-end churn analytics system combining data cleaning, machine learning, LTV modeling, PostgreSQL analytics, FastAPI deployment, Docker containerization, prediction history logging, and Metabase dashboarding.

GitHub Repository: https://github.com/Kapish196258/customer_churn_ltv_system

LinkedIn Project Post: <https://www.linkedin.com/feed/update/urn:li:activity:7476707693374750720>

Table of Contents

<i>S.No.</i>	<i>Section</i>	<i>Page No.</i>
1	Executive Summary	3
2	Project Objectives	3
3	Tools, Libraries and Software Used	3
4	Project Workflow and Execution Hierarchy	4
5	Dataset and Data Preparation	4
6	Data Cleaning and Feature Engineering	5
7	Machine Learning Model Development	5
8	Customer Lifetime Value and Priority Logic	6
9	PostgreSQL Database and Prediction History	6
10	FastAPI Prediction Service	7
11	Dockerization	9
12	Metabase Dashboard and Business Insights	9
13	Source Code and Notebook Inventory	11
14	Repository Structure Summary	12
15	Final Results and Business Interpretation	12
16	Challenges Faced and Learning Outcomes	12
17	Conclusion	13
18	Future Improvements	13
19	Appendix: Screenshot Checklist	13

1. Executive Summary

This report presents an end-to-end Customer Churn Prediction and Customer Lifetime Value (LTV) system. The project was developed to help telecom or subscription-based businesses identify customers who are likely to churn, estimate their value, and prioritize retention actions based on business impact.

The final solution connects multiple parts of a practical data analytics workflow: data cleaning, feature engineering, model training, LTV modeling, PostgreSQL integration, FastAPI prediction service, prediction history logging, Docker containerization, and an interactive Metabase dashboard.

2. Project Objectives

- Predict whether a customer is likely to churn using machine learning.
- Estimate Customer Lifetime Value using monthly charges and tenure.
- Segment customers into Low Value, Medium Value, and High Value groups.
- Identify High Value - High Risk customers for retention priority.
- Store cleaned, processed, LTV, priority, and prediction history data in PostgreSQL.
- Expose the churn prediction model through a FastAPI endpoint.
- Containerize the FastAPI application using Docker for easier deployment.
- Create a Metabase dashboard for business users to analyze churn, LTV, revenue risk, and retention actions

3. Tools, Libraries and Software Used

The project used a combination of programming tools, database systems, machine learning libraries, API frameworks, and business intelligence tools.

Category	Tools / Libraries	Purpose
Programming	Python, SQL	Core data processing, model training, database analysis and backend logic
Development Environment	VS Code, Jupyter Notebook, Command Prompt	Code writing, notebook experimentation, local execution and Docker commands
Data Analysis	Pandas, NumPy	Data cleaning, transformation, feature engineering and calculations
Machine Learning	Scikit-learn, Logistic Regression, Random Forest, XGBoost	Model training, evaluation, scaling and final churn prediction pipeline
Explainability	SHAP	Understanding feature impact on churn prediction
Database	PostgreSQL, pgAdmin, SQLAlchemy, Psycopg2	Data storage, SQL analytics, Python-database integration and prediction history logging
API	FastAPI, Uvicorn, Swagger UI, Pydantic	Prediction API, request validation and browser-based API testing
Dashboard	Metabase	Interactive business dashboard and dashboard filters
Deployment	Docker, Docker Desktop, WSL	Containerized execution of the FastAPI service
Version Control	Git, GitHub	Project tracking, commits, repository submission and documentation

4. Project Workflow and Execution Hierarchy

The project was completed in a structured sequence. Each stage produced a specific output that was used in the next stage.

Step	What Was Done	Output / Deliverable
1	Set up project folders, virtual environment and Git repository	Clean project structure with version control
2	Loaded the raw Telco Customer Churn dataset	Raw dataset placed in data/raw/
3	Cleaned the dataset by fixing TotalCharges, handling missing values and checking duplicates	cleaned_telco_data.csv
4	Performed feature engineering and one-hot encoding	feature_engineered_data.csv
5	Prepared training and testing data with scaling	X_train_scaled.csv, X_test_scaled.csv, y_train.csv, y_test.csv
6	Trained and compared Logistic Regression, Random Forest and XGBoost	Final model selected based on evaluation metrics
7	Saved the final model and later created a scaling + model pipeline	logistic_regression_model.pkl and churn_prediction_pipeline.pkl
8	Performed SHAP explainability	Feature impact understanding for churn prediction
9	Calculated Estimated LTV and customer priority groups	customer_churn_ltv_final.csv and high_priority_customers.csv
10	Loaded business datasets into PostgreSQL and wrote SQL analytics queries	Database tables and dashboard SQL queries
11	Built FastAPI prediction service and tested through Swagger UI	Working /predict endpoint
12	Added prediction history logging into PostgreSQL	prediction_history table and prediction_history.csv export
13	Created Metabase dashboard with LTV Segment filter	Executive and Retention dashboard tabs
14	Dockerized the FastAPI application and tested container execution	Dockerfile, requirements-docker.txt and working Docker image
15	Finalized README, screenshots, report and GitHub submission files	Submission-ready project repository

5. Dataset and Data Preparation

The project uses the Telco Customer Churn dataset. The dataset contains demographic attributes, account details, subscription services, charges, tenure and churn status.

Dataset File	Purpose
data/raw/Telco-Customer-Churn.csv	Original dataset used as the starting point
data/processed/cleaned_telco_data.csv	Cleaned dataset after fixing data types and missing values
data/processed/feature_engineered_data.csv	Model-ready dataset after encoding categorical variables
data/processed/customer_churn_ltv_final.csv	Final analytics dataset with churn, LTV, customer segment, priority and Customer_ID
data/processed/high_priority_customers.csv	Filtered High Value - High Risk customers for retention focus
data/predicted/prediction_history.csv	Export of API prediction history for 10 predicted customers

6. Data Cleaning and Feature Engineering

The raw dataset required cleaning before it could be used for modeling and analytics. The TotalCharges column was converted from text to numeric, missing records were removed, categorical values were validated, and customer identifiers were preserved for later business use.

- TotalCharges was converted to numeric using error handling.
 - Missing TotalCharges records were removed.
 - Duplicate records were checked.
 - Binary categorical variables were mapped to 0 and 1.
 - Multi-category variables were one-hot encoded with drop-first logic.
 - Boolean one-hot columns were converted into integer values.
 - customerID was removed for model training but restored later for business analytics.
- Sample Feature Engineering Logic

```
binary_mapping = {
    "Yes": 1,
    "No": 0,
    "Female": 1,
    "Male": 0
}

df = pd.get_dummies(df, columns=[
    "MultipleLines", "InternetService", "OnlineSecurity",
    "OnlineBackup", "DeviceProtection", "TechSupport",
    "StreamingTV", "StreamingMovies", "Contract", "PaymentMethod"
], drop_first=True)
```

7. Machine Learning Model Development

The churn prediction task was treated as a binary classification problem, where 0 represents no churn and 1 represents churn. Multiple models were trained and evaluated before selecting the final model.

Model	Accuracy	Precision (Churn)	Recall (Churn)	F1-Score (Churn)	Observation
Logistic Regression	80.38%	65%	57%	61%	Best overall balance and selected as final model
Random Forest	79.03%	63%	50%	56%	Good accuracy but lower recall for churned customers
XGBoost	77.83%	59%	55%	57%	Moderate performance but did not improve over Logistic Regression

Logistic Regression was selected because it delivered the strongest overall performance and remained interpretable for a business use case. A final prediction pipeline was created using StandardScaler and Logistic Regression so the API can make consistent predictions.

Saved Model Files

SNo.	Model File Path	Purpose
1	models/saved_model/logistic_regression_model.pkl	Original final Logistic Regression model saved after training
2	models/saved_model/churn_prediction_pipeline.pkl	Final API-ready pipeline containing scaling and Logistic Regression

8. Customer Lifetime Value and Priority Logic

After churn modeling, the project added business value analysis through Customer Lifetime Value. This helped identify not only who may churn, but which customers matter most from a revenue perspective.

Estimated_LTV = MonthlyCharges × tenure

SNo.	Business Metric	Value
1	Total Customers	7,032
2	Churned Customers	1,869
3	Non-Churned Customers	5,163
4	Overall Churn Rate	26.58%
5	High Priority Customers	254
6	Total High-Priority LTV at Risk	1.41 million

LTV Segment	Customer Count	Churn Rate
Low Value	3,516	34.41%
Medium Value	1,758	23.04%
High Value	1,758	14.45%

9. PostgreSQL Database and Prediction History

PostgreSQL was used to store cleaned data, final LTV analytics data, high-priority customer data and prediction history. This made the project more practical because the outputs were not only stored in CSV files but also available for SQL analysis and dashboarding.

SNo	Table Name	Purpose
1	telco_customer_churn	Cleaned customer churn dataset stored in PostgreSQL
2	customer_churn_ltv	Final LTV-enhanced analytics table with Customer_ID, LTV segment and priority
3	high_priority_customers	High Value - High Risk customers for retention targeting
4	prediction_history	Stores customer-level API predictions, probability, estimated LTV and timestamp

Prediction History Flow

1. User enters model-ready customer values in FastAPI Swagger UI.
2. The /predict endpoint sends the values into the churn prediction pipeline.
3. The model returns churn prediction and churn probability.
4. Estimated LTV is calculated using $\text{MonthlyCharges} \times \text{tenure}$.
5. The result is stored in PostgreSQL prediction_history table.
6. A CSV export of 10 prediction records is stored in data/predicted/prediction_history.csv.

Prediction History Table

S.No.	Customer ID	Churn Prediction	Churn Meaning	Churn Probability	Estimated LTV	History Logged
1	7590-VHVEG	1	Likely to Churn	0.7505	118.75	True
2	5575-GNVDE	0	Not Likely to Churn	0.0072	2860.80	True
3	3668-QPYBK	1	Likely to Churn	0.7576	180.70	True
4	7795-CFOCW	0	Not Likely to Churn	0.0013	1836.00	True
5	9237-HQITU	1	Likely to Churn	0.8168	398.60	True
6	9305-CDSKC	0	Not Likely to Churn	0.2458	2034.00	True
7	1452-KIOVK	1	Likely to Churn	0.7927	661.50	True
8	6713-OKOMC	0	Not Likely to Churn	0.0062	2688.00	True
9	7892-POOKP	1	Likely to Churn	0.7543	838.40	True
10	6388-TABGU	0	Not Likely to Churn	0.0324	2516.40	True

10. FastAPI Prediction Service

FastAPI was used to expose the churn model as a prediction API. The endpoint accepts model-ready encoded numerical input and returns churn prediction, churn probability, estimated LTV and prediction history logging status.

API Detail	Value
Local Swagger URL	http://127.0.0.1:8000/docs
Docker Swagger URL	http://localhost:8000/docs
Endpoint	POST /predict
Main API File	api/main.py
Schema File	api/schemas.py
Prediction Logic	src/models/churn_predict.py
History Logging	src/database/prediction_history.py

Sample API Response

```
{
  "prediction": 1,
  "churn_probability": 0.7505,
  "estimated_ltv": 118.75,
  "history_logged": true
}
```

Figure 1: FastAPI Swagger Input

Customer Churn Prediction API

1.0.0 OAS 3.1

[/openapi.json](#)

FastAPI service for predicting customer churn using model-ready encoded numerical input features.

default

GET / Home

POST /predict Predict

Parameters

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "customer_id": "7590-WVEG",
  "gender": 1,
  "SeniorCitizen": 1,
  "Partner": 0,
  "Dependents": 0,
  "tenure": 1,
  "PhoneService": 1,
  "PaperlessBilling": 1,
  "MonthlyCharges": 118.75,
  "TotalCharges": 118.75,
  "MultipleLines_No_phone_service": 0,
  "MultipleLines_Yes": 1,
  "InternetService_Fiber_optic": 1,
  "InternetService_No": 0,
  "OnlineSecurity_No_internet_service": 0,
  "OnlineSecurity_Yes": 0,
  "OnlineBackup_No_internet_service": 0,
  "OnlineBackup_Yes": 0,
  "DeviceProtection_No_internet_service": 0,
  "DeviceProtection_Yes": 0,
  "TechSupport_No_internet_service": 0,
  "TechSupport_Yes": 0,
  "StreamingTV_No_internet_service": 0,
  "StreamingTV_Yes": 1
}
```

Figure 2: FastAPI Prediction Result

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "customer_id": "7590-WVEG",
    "gender": 1,
    "SeniorCitizen": 1,
    "Partner": 0,
    "Dependents": 0,
    "tenure": 1,
    "PhoneService": 1,
    "PaperlessBilling": 1,
    "MonthlyCharges": 118.75,
    "TotalCharges": 118.75,
    "MultipleLines_No_phone_service": 0,
    "MultipleLines_Yes": 1,
    "InternetService_Fiber_optic": 1,
    "InternetService_No": 0,
    "OnlineSecurity_No_internet_service": 0,
    "OnlineSecurity_Yes": 0,
    "OnlineBackup_No_internet_service": 0,
    "OnlineBackup_Yes": 0,
    "DeviceProtection_No_internet_service": 0,
    "DeviceProtection_Yes": 0,
    "TechSupport_No_internet_service": 0,
    "TechSupport_Yes": 0,
    "StreamingTV_No_internet_service": 0,
    "StreamingTV_Yes": 1
  }'
```

Request URL

http://127.0.0.1:8000/predict

Server response

Code

Details

200

Response body

```
{
  "prediction": 1,
  "churn_probability": 0.7505,
  "estimated_ltv": 118.75,
  "history_logged": true
}
```

Response headers

```
content-length: 88
content-type: application/json
date: Sat, 27 Jun 2026 18:08:16 GMT
server: unicorn
```


11. Dockerization

Docker was added so the FastAPI service can run in a consistent containerized environment. This avoids dependency conflicts and makes the API easier to run on another system.

Docker File	Purpose
Dockerfile	Defines the container image using Python 3.11 slim, installs runtime dependencies, copies project files and starts Uvicorn
.dockerignore	Prevents unnecessary files such as venv, .env, Git files and cache files from being copied into the image
requirements-docker.txt	Contains only the runtime packages required for the FastAPI Docker container

Docker Commands Used

```
docker build -t customer-churn-ltv-api .
docker run --rm -p 8000:8000 customer-churn-ltv-api
```

12. Metabase Dashboard and Business Insights

Metabase was used to present the final business insights in a dashboard format. The dashboard was designed for non-technical users so they can understand churn risk, customer value and retention priorities without reading code.

- Executive Overview tab: overall KPIs, churn distribution, churn rate by LTV segment and revenue value by priority group.
- Retention Action & Customer Insights tab: retention priority summary, segment-level revenue impact and customer-level retention lists.
- LTV Segment filter: allows users to filter selected dashboard cards by Low Value, Medium Value and High Value segments.

Dashboard Component	Purpose
Total Customers	Shows total customer base used for analysis
Overall Churn Rate	Shows percentage of churned customers
Total LTV at Risk	Shows revenue value exposed to churn risk
High Priority Customers	Shows count of High Value - High Risk customers
Retention Priority Summary	Helps business team decide where to focus retention actions
Top Customers to Retain	Provides customer-level action list

Figure 3: Executive Overview Dashboard

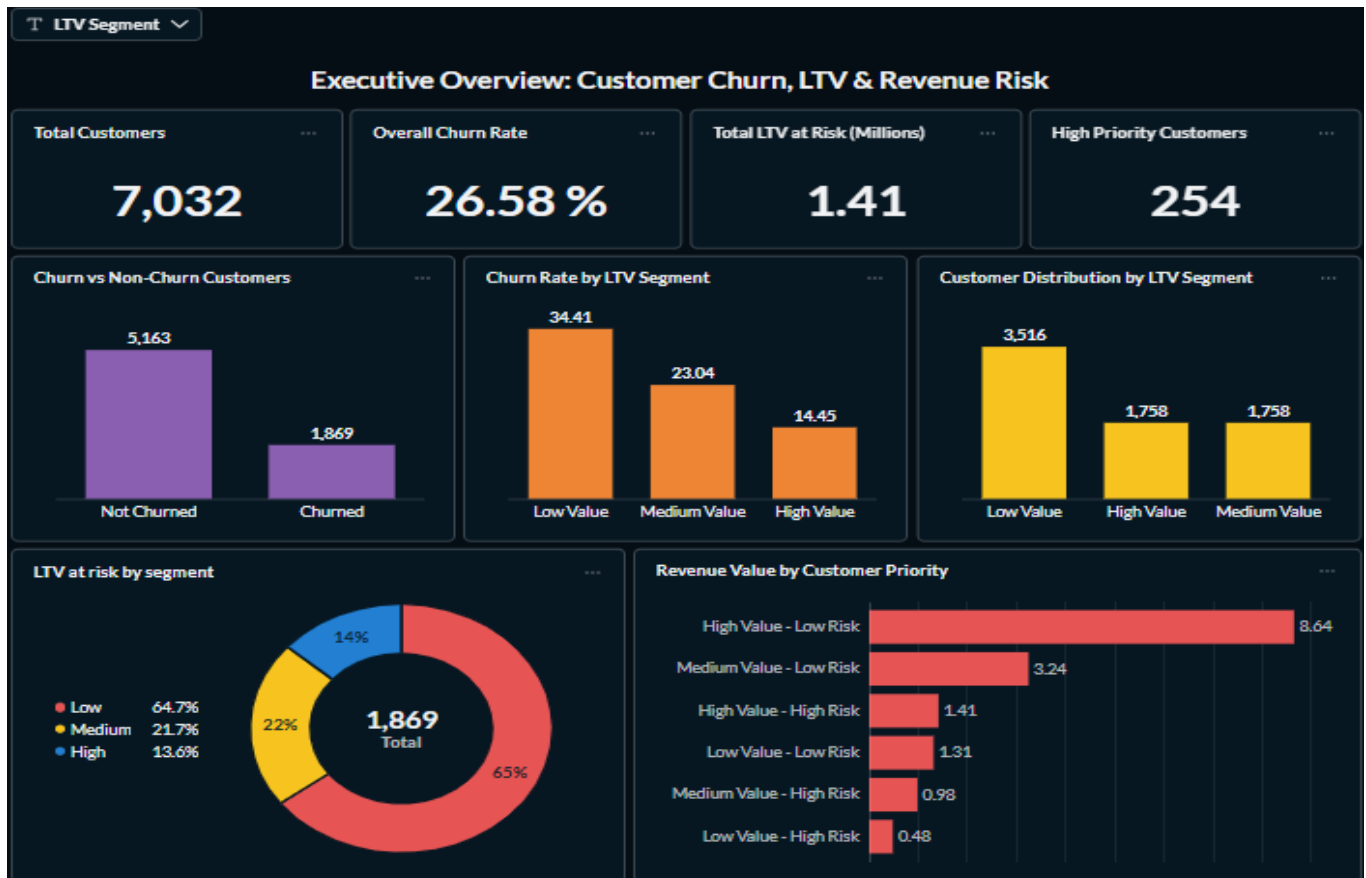


Figure 4: Retention Action & Customer Insights Dashboard

T

LTV Segment

▼

Retention Action & Customer Insights					
Retention Priority Summary ⓘ					
Customer Priority	Customers	Avg Monthly Charges	Avg LTV	avg_tenure_months	
High Value - Low Risk	1,504	92.63	5,747.11	62	
High Value - High Risk	254	99.49	5,534.91	56	
Medium Value - High Risk	405	84.71	2,422.89	30	
Medium Value - Low Risk	1,353	63.98	2,393.33	42	
Low Value - Low Risk	2,306	39.31	568.38	19	
Low Value - High Risk	1,210	65.75	392.93	6	
6 rows					
Segment-wise Churn and Revenue Impact ⓘ					
LTV_Segment	total_customers	churned_customers	churn_rate_percentage	total_ltv_million	churned_ltv_million
High Value	1,758	254	14.45	10.05	1.41
Medium Value	1,758	405	23.04	4.22	0.98
Low Value	3,516	1,210	34.41	1.79	0.48
3 rows					

13. Source Code and Notebook Inventory

The repository was organized so that notebooks show experimentation and analysis, while Python files contain reusable project logic.

Python Source Files

File	Purpose
api/main.py	Creates the FastAPI app, home route and /predict endpoint
api/schemas.py	Defines API input fields, validation and Swagger descriptions
api/routes.py	Reserved for modular API routing if the project is expanded
src/database/connection.py	Handles PostgreSQL database connection logic
src/database/prediction_history.py	Saves API prediction results into PostgreSQL
src/preprocessing/clean_data.py	Loads and cleans the raw Telco dataset
src/preprocessing/featureengineering.py	Encodes categorical variables and prepares model features
src/preprocessing/ltv_calculator.py	Calculates estimated LTV and customer segments
src/models/churn_predict.py	Loads the churn prediction pipeline and returns prediction output
src/models/train_churn_pipeline.py	Trains the scaler + Logistic Regression pipeline for API use
src/models/ltv_predict.py	Supports LTV-related prediction or calculation workflow
src/models/shap_analysis.py	Supports SHAP-based model explainability
src/utils/config.py	Stores reusable paths, constants and environment configuration
dashboard/dashboard_sql_queries.py	Stores reusable SQL queries used in dashboard cards

Jupyter Notebooks

Notebook	Purpose
eda.ipynb	Exploratory data analysis
data_cleaning.ipynb	Data cleaning and validation
feature_engineering.ipynb	Encoding and feature creation
model_preparation.ipynb	Train-test split and scaling
model_training.ipynb	Model training and comparison
shap_analysis.ipynb	Model explainability
ltv_modelling.ipynb	Customer Lifetime Value modeling
ltv_postgresql_integration.ipynb	Loading LTV data into PostgreSQL
ltv_postgresql_analysis.ipynb	SQL analysis for LTV and churn
customer_data_enrichment.ipynb	Restoring and enriching customer-level identifiers

14. Repository Structure Summary

The full repository contains API code, source modules, notebooks, processed datasets, saved models, dashboard queries, screenshots, Docker files and documentation. The most important folders are summarized below.

customer_churn_ltv_system/	
├─ api/	FastAPI app, schema and API screenshots
├─ dashboard/	Metabase SQL queries and dashboard screenshots
├─ data/raw/	Original Telco dataset
├─ data/processed/	Cleaned, engineered, LTV and high-priority CSV files
├─ data/predicted/	prediction_history.csv export
├─ models/saved_model/	Saved ML model and API-ready pipeline
├─ notebooks/	Jupyter notebooks for analysis and experiments
├─ src/database/	PostgreSQL connection and prediction history logging
├─ src/preprocessing/	Data cleaning, feature engineering and LTV modules
├─ src/models/	Model prediction, training pipeline and SHAP logic
├─ Dockerfile	Docker image recipe for FastAPI
├─ requirements-docker.txt	Docker runtime dependencies
└─ README.md	Final project documentation

15. Final Results and Business Interpretation

The final system helps identify churn risk and customer value together. This is important because not every churned customer has the same business impact. By combining churn prediction with LTV, the business can prioritize the customers whose loss would create the highest revenue risk.

- The overall churn rate was 26.58%, showing a meaningful retention challenge.
- Low Value customers had the highest churn rate, but High Value - High Risk customers were the most important retention group.
- 254 customers were identified as High Value - High Risk.
- The total estimated LTV at risk among high-priority customers was approximately 1.41 million.
- The FastAPI endpoint allows new customer-level predictions to be generated and stored.
- The Metabase dashboard converts technical outputs into business-readable insights.

16. Challenges Faced and Learning Outcomes

The project required connecting multiple technical components rather than building only a notebook-based model. The main challenges were maintaining consistent feature order, handling scaled model input, restoring customer identifiers for business reporting, integrating PostgreSQL, building a clean API response, and Dockerizing the FastAPI application.

Through this project, I learned how to move from raw data to a full analytics solution: cleaning data, preparing features, training models, explaining predictions, calculating LTV, storing data in PostgreSQL, exposing a prediction API, logging prediction history, creating dashboards, and preparing a deployment-ready Docker container.

17. Conclusion

This project successfully demonstrates a complete Customer Churn Prediction and LTV analytics workflow. It combines machine learning, business analytics, PostgreSQL, FastAPI, Docker and Metabase into one practical system for customer retention decision-making.

The final solution can predict churn, estimate customer value, identify retention priorities, store prediction history, and present insights through dashboards. This makes it useful for both technical evaluation and business decision-making.

18. Future Improvements

- Deploy the FastAPI service to a cloud platform.
- Deploy the Metabase dashboard for shared business access.
- Add automated model retraining when new customer data is available.
- Add CI/CD for automated testing and deployment.
- Add authentication and monitoring for production API usage.

Appendix: Screenshot Checklist

Recommended screenshots to include before final submission:

- FastAPI Swagger input screen showing encoded field descriptions.
- FastAPI prediction response showing prediction, churn probability, estimated LTV and history logging status.
- PostgreSQL prediction_history table with sample predictions.
- Executive Overview dashboard screenshot.
- Retention Action & Customer Insights dashboard screenshot.
- Docker container running successfully or Docker command output, if required.